

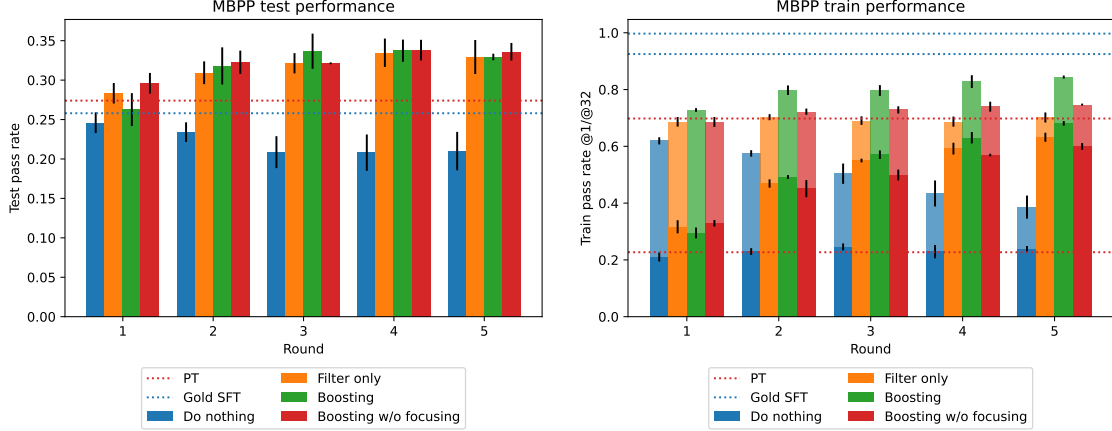


Figure 2: We plot test and train performance of our Algorithm 2 variants on MBPP, across rounds. We report the mean and `np.std(*,ddof=1)` for 3 seeds. For train pass rate plots, we plot both train pass@1 (solid) and train pass@32 (stacked). Boosting results displayed here use *weak data (A)*.

than giving the labeler the prompts we got wrong, P_t^- , we draw a random set of questions of size $|P_t^-|$.

We also report two baselines that do not involve iteratively training on model-generated data. **PT**: the pre-trained model; and **Gold SFT**: the model after one round of fine-tuning on the human-written responses in the dataset. Note that *Gold SFT* is the only setup that makes use of human-written responses, rather than just for answer verification.

7.2 Experimental Details

In all experiments, a round of fine-tuning entails training all parameters of the model for 330 (GSM8K) or 30 (MBPP) steps at batch size 64 (with the exception of *Gold SFT* where we report the checkpoint with best validation accuracy) We train with standard sequence cross-entropy loss. Training examples are (`input`, `target`) pairs, where `input` is the problem preceded by a 3-shot prompt (see Appendix D for prompt templates); and `target` is a model response (human-written response for *Gold SFT*).

Modeling the weak data. We instantiate `labeler` as a Gemma 2 2B PT model with a fixed *total query budget*, which is distributed uniformly over all problems it receives. For a given problem, we sample responses from the model equal to that problem’s allotted queries. We consider two setups to simulate weak data provided by the weak labeler.

- **Weak data (A)**: for each question, we return all correct responses if there are any. If there are none, we return a random incorrect response.
- **Weaker data (B)**: we pool together the correct responses to all questions. We add to this collection an equal number of incorrect responses, drawn randomly from all incorrect responses to all questions.

Setup	Rounds	GSM8K			MBPP		
		train		test	train		test
		@1	@8	greedy	@1	@32	greedy
PT	0	.211. _{.002}	.630. _{.007}	.222. _{.003}	.235. _{.008}	.703. _{.007}	.275. _{.002}
- orig. report		-	-	.243*	-	-	.302*
Gold SFT	1	.392. _{.020}	.755. _{.009}	.379. _{.008}	.880. _{.039}	.987. _{.009}	.237. _{.018}
Do nothing	5	.136. _{.012}	.391. _{.026}	.129. _{.022}	.238. _{.011}	.386. _{.041}	.210. _{.024}
Filter only	5	.553. _{.050}	.762. _{.029}	.393. _{.020}	.632. _{.016}	.702. _{.018}	.329. _{.022}
Boosting (A)	5	.589. _{.013}	.849. _{.005}	.456. _{.010}	.681. _{.008}	.844. _{.005}	.329. _{.004}
- w/o focusing		.550. _{.008}	.787. _{.005}	.432. _{.013}	.600. _{.012}	.747. _{.003}	.336. _{.011}
Boosting (B)	5	.565. _{.020}	.820. _{.009}	.443. _{.020}	.647. _{.012}	.832. _{.009}	.326. _{.009}
- w/o focusing		.509. _{.000}	.767. _{.007}	.430. _{.012}	.544. _{.012}	.698. _{.010}	.327. _{.009}

Table 1: Comparison of 3-shot train and test accuracy@ k rates on GSM8K and MBPP for Gemma 2 2B checkpoints produced by various setups. We report the mean and `np.std(*, ddof=1)` for 3 seeds. To report train accuracy@ k , we sample k solutions to each problem at temperature 0.7 and mark it correct if any of k solutions is correct. For test accuracy, we employ greedy sampling. (*): Row 2 cites the figure from the Gemma 2 report [Gemma Team et al., 2024] which does not report sampling temperature.

We remark that the fixed total query budget setup offers a mechanism for satisfying the weak data assumption: the labeler can maintain constant accuracy when targeting increasingly granular (and more difficult) slices of the input distribution by focusing their resources. We see that this is indeed the case experimentally, and plot accuracies in Figure 3. Moreover, a fixed query budget is a natural analogue to the fixed person-hours/money/compute budgets behind a labelling effort.

Departure from the theory. In our experiments, we make one main modification from Algorithm 2. Rather than accumulating *data* and retraining the model each iteration (Algorithm 2, line 9), we instead accumulate *updates*. That is, we fine-tune on the newly introduced data in each iteration, initializing from the checkpoint produced by the prior iteration. We do this for efficiency reasons.

7.3 GSM8K Results

Table 1 (first half) summarizes our results on GSM8K. We have 7000 training problems, use $k = 8$ for `generate`, and allocate the same total query budget of 56,000 to the labeler each round. In Appendix C, we present model responses to selected problems over the course of training.

Baselines validate our experimental setup. Results in the *PT* and *Gold SFT* demonstrate that: (1) our evaluation setup is in the ballpark of what is reported in the original Gemma 2 report; and (2) our fine-tuning setup indeed can yield significant improvement when the training data is human-written solutions.

Model collapse with no curation. In the *Do nothing* row, we recover the result from the model collapse literature that iterative fine-tuning without curation does not improve the model and leads to degraded quality.

Comparison between curation variants. Indeed, the present algorithm demonstrates improvements over the ReST-like variant that uses filtering only. The differences are most evident in training accuracy, which is strongly predicted by the theory. Indeed, this is in spite of the fact that *as opposed to filtering only, boosting introduces incorrect answers to the training data*. Furthermore although our theory does not address generalization, we observe that boosting results in improved test accuracy. Finally, the performance of boosting without focusing is quite close – random selection is a strong baseline – but focusing still leads to improvements, especially in terms of training accuracy.

7.4 MBPP Results

Table 1 (second half) summarizes our results on MBPP. We have 374 training problems, use $k = 32$ for **generate**, and allocate the same total query budget of 11,968 to the weak labeler in each round.

Similar results to GSM8K for train pass rate. In terms of train pass@ k , we observe similar results to GSM8K experiments, that generally: *Boosting* > *Boosting w/o focusing* > *Filter only* > *Do nothing*. On *weaker data (B)*, *Filtering* beats *Boosting w/o focusing* in terms of pass@1.

No clear winner for test pass rate. While all iterative approaches outperform *Gold SFT* in terms of test pass rate, they all recover similar test performance despite differences in training accuracy. Notably, *Boosting w/o focusing* beats *Boosting*, and *Filter Only* outperforms *Boosting* with weaker data (B). One explanation is the limited amount of training data (384 examples) which prevents generalization; note that *Gold SFT* does not recover *PT* test pass rate.

8 Conclusion & Future Work

We have shown that under mild assumptions a modicum of curation applied to synthetic data not only avoids model collapse, but leads to arbitrarily high accuracy results. Our analysis is through the lens of boosting and, mirroring that paradigm, we define notions of *strong learners* and *weak data* to reach the theoretical conclusions. In taking this view, we provide theoretical explanations for many of the synthetic data methods used in practice.

Many interesting questions remain. An immediate avenue is further relaxing the assumptions (e.g., having nearly strong learners that only approximately match the conditional distribution) and deriving corresponding convergence rates. A broader goal is using these insights for the burgeoning field of data selection, where we must explicitly model similarities between different examples as part of the analysis.

Author contributions

- **Sergei V** conceived the idea to analyze synthetic data training through the lens of boosting.